Orchestrator — query decomposition, cortex dispatch, and result assembly

SPEC-TH-001 v0.1 DRAFT Jeremie Nunez

1 Context

The Thalamus orchestrator is the top-level coordinator of the multi-cortex research agent. It receives a user query, decomposes it into a plan of cortex invocations, dispatches those invocations (in parallel across cortices, sequentially within a cortex chain), merges structured findings, and streams events to the caller. The orchestrator is deterministic glue around three moving parts: the *planner* (LLM, selects cortices and steps), the *cortex executor* (runs a cortex, see SPEC-TH-003), and the *result assembler* (normalises, deduplicates, and streams findings).

2 Goals

- Turn a single query into a plan of N missions (one per selected cortex) with well-formed steps.
- Execute missions in parallel while preserving per-mission step order and chaining (`pickFrom` / `feedsInto`).
- Emit a typed event stream (`tool_start`, `tool_result`, `tool_error`, `agent_done`) that the UI or upstream caller can consume incrementally.
- Enforce mutation guards: tools that mutate state only run when the query intent explicitly authorises mutation.
- Enforce per-tool timeouts and make a failed step non-fatal for the remainder of its mission.

3 Non-Goals

Non-Goal

The orchestrator does not implement domain logic (scoring, SQL, LLM reasoning) — that is owned by each cortex (SPEC-TH-003). It does not persist findings to the knowledge graph; the caller is responsible for consuming the event stream and committing. It does not negotiate budget targets across iterations; that loop lives one level above.

4 Invariants

Invariant

For every mission `M` and every step `S` in `M`, exactly one of `tool_result` or `tool_error` is emitted, and it references a `toolCallId` previously seen in a `tool_start`. Exactly one `agent_done` is emitted per mission, after all its step events.

Invariant

A mutating tool is dispatched if and only if the caller's intent is in the set of action intents. A blocked mutation yields a `tool_error` with reason `mutation_blocked` and does not call the tool.

Invariant

A step timeout or thrown exception in one step never prevents subsequent steps in the same mission from being attempted, and never cancels sibling missions.

5 Interface

Public surface consumed by the HTTP layer and the recursive research loop.

```
export interface Mission {
  agent: AgentRole;
  steps: MissionStep[];
}

export interface MissionStep {
  tool: string;
  params: Record<string, unknown>;
  pickFrom?: PickStrategy;       // select from previous result array
  feedsInto?: string;            // param name fed by previous id(s)
  size?: CardSize;               // UI hint only
}

export type PipelineSSEEvent =
  | ToolStartEvent
  | ToolResultEvent
  | ToolErrorEvent
  | AgentDoneEvent;

export class MissionExecutor {
  constructor(toolMap: Map<string, DynamicStructuredTool>);

  executeAllMissions(
    missions: Mission[],
    intent: string,
  ): AsyncGenerator<PipelineSSEEvent>;

  executeMission(
    mission: Mission,
    intent: string,
  ): AsyncGenerator<PipelineSSEEvent>;

  retrySingleTool(
    toolName: string,
    modifiedInput: Record<string, unknown>,
    agent?: AgentRole,
  ): AsyncGenerator<PipelineSSEEvent>;
}
```

6 Scenarios

Scenario: Nominal parallel dispatch

Given a plan of three missions `[M1, M2, M3]` and a read-only intent
When `executeAllMissions` is consumed to completion
Then step events from the three missions may interleave in time order

And each mission emits exactly one `agent_done` after its last step
And the total number of `tool_start` events equals the total number of steps.

Scenario: Step chaining via pickFrom/feedsInto

Given a mission whose step 1 returns `{ items: [{id:7}, {id:9}] }`
And step 2 declares `pickFrom: "top_by_relevance"` and `feedsInto: "itemId"`
When the mission is executed
Then step 2 is invoked with `params.itemId === 7`
And no `tool_error` is emitted for step 2 due to chaining.

Scenario: Mutation blocked on read intent

Given a mission whose step invokes a mutating tool
And the caller intent is not in the action intents set
When the mission is executed
Then a single `tool_error` with reason `mutation_blocked` is emitted for that step
And the tool is never invoked
And subsequent steps in the mission still run.

Scenario: Tool timeout is recoverable

Given a mission with three steps where step 2 exceeds its timeout
When the mission is executed
Then step 2 emits `tool_error` with a timeout message
And step 3 still emits `tool_start` and terminates normally
And sibling missions are unaffected.

Scenario: Empty upstream breaks the chain

Given step 1 returns an empty result array and step 2 declares `pickFrom`
When the mission is executed
Then step 2 emits `tool_error` with reason `no_results_from_previous_step`
And no further steps of that mission are attempted.

7 Acceptance Criteria

Acceptance Criterion AC-1

Given a plan of n missions, `executeAllMissions` yields exactly n `agent_done` events; for each mission the `agent_done` is the last event bearing that `agent` field.

Acceptance Criterion AC-2

For every step, the `toolCallId` appearing in its `tool_start` matches the `toolCallId` of its terminal `tool_result` or `tool_error`.

Acceptance Criterion AC-3

A step marked as using a mutating tool, under a non-action intent, produces a `tool_error` with code `mutation_blocked` and does not invoke the tool (verified via a spy).

Acceptance Criterion AC-4

A step whose underlying tool rejects or exceeds the applicable timeout emits exactly one `tool_error` and does not prevent later steps of the same mission from emitting.

Acceptance Criterion AC-5

Chaining with `pickFrom: "top_by_relevance"` feeds `results[0].id` into the declared `feedsInto` parameter of the next step.

Acceptance Criterion AC-6

An exception thrown by a tool in mission M_i does not cancel or short-circuit mission M_j for $j \neq i$: both missions still emit an `agent_done`.

Acceptance Criterion AC-7

`retrySingleTool` invoked with a tool in the retry-blocked set emits a single `tool_error` and does not invoke the tool.

8 Traceability

AC	Test file	Test name
AC-1	`packages/thalamus/tests/orchestrator.spec.ts`	`emits one agent_done per mission`
AC-2	`packages/thalamus/tests/orchestrator.spec.ts`	`toolCallId round-trip`
AC-3	`packages/thalamus/tests/orchestrator.spec.ts`	`mutation blocked on read intent`
AC-4	`packages/thalamus/tests/orchestrator.spec.ts`	`timeout does not abort mission`
AC-5	`packages/thalamus/tests/orchestrator.spec.ts`	`pickFrom feeds top result id`
AC-6	`packages/thalamus/tests/orchestrator.spec.ts`	`sibling mission isolation`
AC-7	`packages/thalamus/tests/orchestrator.spec.ts`	`retry blocks disallowed tools`

9 Open Questions

- Should `agent_done` carry per-mission aggregate metrics (step count, error count) or remain a pure marker?
- Should chaining support multi-source fan-in (step k reads from step $k-2$), or is single-predecessor sufficient for v1?
- Where is the hard cap on concurrent missions enforced — orchestrator, caller, or transport layer?